Block Matchers

With all the expectations we've seen so far, we've passed regular Ruby objects into expect:

11-matchers-included-in-rspec-expectations/03/block_matchers.rb
```
expect(3).to eq(3)
```

This is fine for checking properties of your data. But sometimes you need to check properties of a *piece of code*. For example, perhaps a certain piece of code is supposed to raise an exception. In this case, you can pass a block into expect:

11-matchers-included-in-rspec-expectations/03/block_matchers.rb
```
expect { raise 'boom' }.to raise_error('boom')
```

RSpec will run the block and watch for the specific side effects you specify: exceptions, mutating variables, I/O, and so on.

Raising and Throwing

You're likely familiar with raising Ruby exceptions to jump out of your running code and report an error to the caller. Ruby also has a related concept, throwing symbols, for jumping to other parts of your program.

RSpec provides matchers for both of these situations: the appropriately named raise_error and throw_symbol.

raise_error

First, let's look at raise_error, also known as raise_exception. This matcher is very flexible, supporting multiple forms:

- raise_error with no arguments matches if *any* error is raised.
- raise_error(SomeErrorClass) matches if SomeErrorClass or a subclass is raised.
- raise_error('some message') matches if an error is raised with a message exactly equal to a given string.
- raise_error(/some regex/) matches if an error is raised with a message matching a given pattern.

You can combine these criteria if both the class and message are important, either via passing two arguments or by using a fluent interface:

- raise_error(SomeErrorClass, "some message")
- raise_error(SomeErrorClass, /some regex/)
- raise_error(SomeErrorClass).with_message("some message")
- raise_error(SomeErrorClass).with_message(/some regex/)